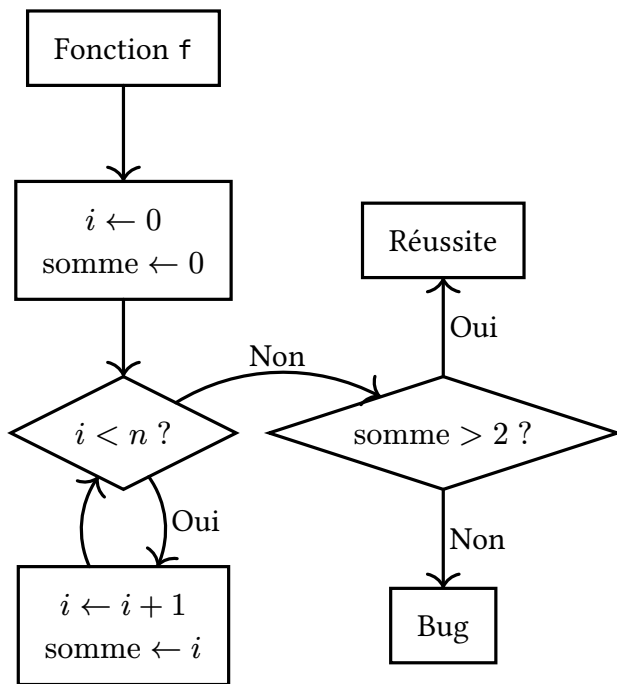




Aujourd'hui nous avons joué avec des **graphes de flot de contrôle** représentant des recettes de cuisine pour faire des confitures. Ils contiennent des **instructions** et des **conditions**, portant sur des **variables**. Ces graphes ont pu être plus ou moins complexes, avec notamment des **boucles**.

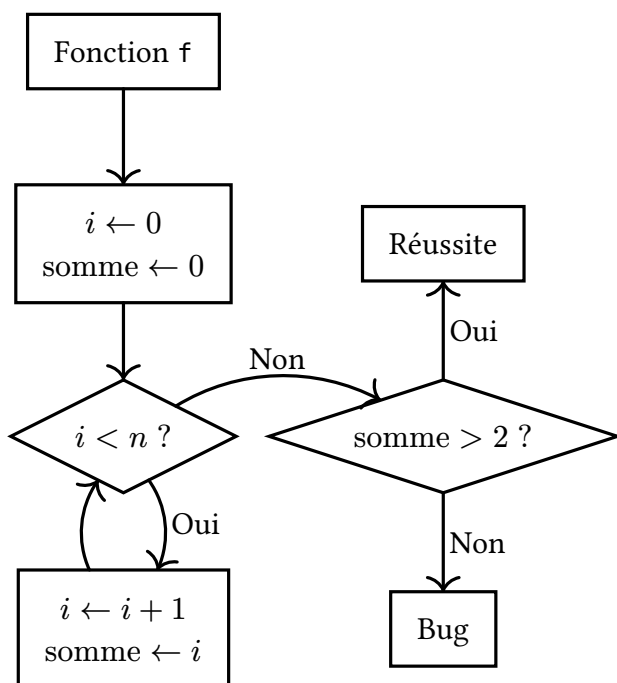
Ces graphes représentent en réalité des **algorithmes**, qui comme pour les recettes, peuvent réussir ou échouer : on parle alors de **bugs**.

Afin d'éviter ces bugs, les informaticiens mettent en place des **tests**, programmes exécutés par eux et non par les utilisateurs, afin d'essayer de prouver que leurs programmes ne possèdent pas de bugs.

Pour mesurer la qualité de ces tests, ils s'intéressent à la **couverture** de ces tests, c'est à dire à quels point ces tests passent par toutes les instructions et conditions (**couverture par sommet**) ou par toutes les flèches (**couverture**

par arêtes). Cependant, rien n'assure dans cette méthode l'absence totale de bugs.

Pour être sûr qu'il n'y a pas de bug, certains informaticiens peuvent utiliser de l'**exécution symbolique** afin de chercher depuis un bug quelles entrées les causent. Cependant, cette technique demande de résoudre des grands systèmes d'équations, ce qui est lent. C'est pourquoi cette méthode n'est pas souvent utilisée.



Aujourd'hui nous avons joué avec des **graphes de flot de contrôle** représentant des recettes de cuisine pour faire des confitures. Ils contiennent des **instructions** et des **conditions**, portant sur des **variables**. Ces graphes ont pu être plus ou moins complexes, avec notamment des **boucles**.

Ces graphes représentent en réalité des **algorithmes**, qui comme pour les recettes, peuvent réussir ou échouer : on parle alors de **bugs**.

Afin d'éviter ces bugs, les informaticiens mettent en place des **tests**, programmes exécutés par eux et non par les utilisateurs, afin d'essayer de prouver que leurs programmes ne possèdent pas de bugs.

Pour mesurer la qualité de ces tests, ils s'intéressent à la **couverture** de ces tests, c'est à dire à quels point ces tests passent par toutes les instructions et conditions (**couverture par sommet**) ou par toutes les flèches (**couverture**

par arêtes). Cependant, rien n'assure dans cette méthode l'absence totale de bugs.

Pour être sûr qu'il n'y a pas de bug, certains informaticiens peuvent utiliser de l'**exécution symbolique** afin de chercher depuis un bug quelles entrées les causent. Cependant, cette technique demande de résoudre des grands systèmes d'équations, ce qui est lent. C'est pourquoi cette méthode n'est pas souvent utilisée.